

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

**образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

Отчет по лабораторной работе №2

По дисциплине “Информационные технологии и программирование”

Выполнил: студент Петунин Алексей
группы БПИ2401

Проверил: Харрасов Камиль Раисович

Москва

2025

Цель работы

Изучение и практическая реализация четырех основных принципов объектно-ориентированного программирования: инкапсуляции, наследования, полиморфизма и абстракции.

Задание.

Абстрактный класс (Нельзя создать объект, только наследовать):

```
J Bicycle.java X
J Bicycle.java > Language Support for Java(TM) by Red Hat > Bicycle > displayInfo()
1 public abstract class Bicycle {
2     private String brand;
3     private int wheelSize;
4     private String color;
5     private static int count = 0;
6
7     public Bicycle(String brand, int wheelSize, String color) {
8         this.brand = brand;
9         this.wheelSize = wheelSize;
10        this.color = color;
11        count++;
12    }
13
14    public Bicycle() { this(brand:"Unknown", wheelSize:20, color:"Black"); }
15
16    public abstract void ride();
17    public abstract void brake();
18
19    public String getBrand() { return brand; }
20    public void setBrand(String brand) { this.brand = brand; }
21    public int getWheelSize() { return wheelSize; }
22    public void setWheelSize(int wheelSize) { this.wheelSize = wheelSize; }
23    public String getColor() { return color; }
24    public void setColor(String color) { this.color = color; }
25    public static int getCount() { return count; }
26
27    public void displayInfo() {
28        System.out.println(brand + ", колеса: " + wheelSize + ", цвет: " + color);
29    }
30 }
```

Конкретный класс (Нельзя создать объект, только наследовать):

```
J MountainBike.java 2 X
J MountainBike.java > Java > MountainBike
1 public class MountainBike extends Bicycle {
2     private int gears;
3     private boolean suspension;
4
5     public MountainBike(String brand, int wheelSize, String color, int gears, boolean suspension) {
6         super(brand, wheelSize, color);
7         this.gears = gears;
8         this.suspension = suspension;
9     }
10
11    public MountainBike() { super(); this.gears = 21; this.suspension = true; }
12
13    @Override
14    public void ride() {
15        System.out.println("Еду по горам на " + getBrand() + " (" + gears + " передач)");
16    }
17
18    @Override
19    public void brake() {
20        System.out.println("Торможу дисковыми тормозами");
21    }
22
23    public int getGears() { return gears; }
24    public boolean hasSuspension() { return suspension; }
25 }
```

Конкретный класс (Можно создавать объекты, полная реализация):

```
J KidsBike.java 2 X
J KidsBike.java > Language Support for Java(TM) by Red Hat > KidsBike
1 public class KidsBike extends Bicycle {
2     private boolean trainingWheels;
3     private int maxAge;
4
5     public KidsBike(String brand, int wheelSize, String color, boolean trainingWheels, int maxAge) {
6         super(brand, wheelSize, color);
7         this.trainingWheels = trainingWheels;
8         this.maxAge = maxAge;
9     }
10
11     public KidsBike() { super(); this.trainingWheels = true; this.maxAge = 8; }
12
13     @Override
14     public void ride() {
15         System.out.println("Детский велосипед " + getBrand() + " едет безопасно");
16     }
17
18     @Override
19     public void brake() {
20         System.out.println("Торможу ножным тормозом");
21     }
22
23     public boolean hasTrainingWheels() { return trainingWheels; }
24     public int getMaxAge() { return maxAge; }
25 }
```

Конкретный класс (Можно создавать объекты, полная реализация):

```
J BMX.java 2 X
J BMX.java > Language Support for Java(TM) by Red Hat > BMX
1 public class BMX extends Bicycle {
2     private boolean pegs;
3     private String style;
4
5     public BMX(String brand, int wheelSize, String color, boolean pegs, String style) {
6         super(brand, wheelSize, color);
7         this.pegs = pegs;
8         this.style = style;
9     }
10
11     public BMX() { super(); this.pegs = true; this.style = "Street"; }
12
13     @Override
14     public void ride() {
15         System.out.println("Выполняю трюки в стиле " + style + " на BMX");
16     }
17
18     @Override
19     public void brake() {
20         System.out.println("Торможу для трюка");
21     }
22
23     public boolean hasPegs() { return pegs; }
24     public String getStyle() { return style; }
25 }
```

Классы MountainBike, KidsBike и BMX наследуют от базового класса Bicycle, получая общие свойства и поведение.

Главный класс (Содержит точку входа (main)):

```
J Main.java 2 x
J Main.java > Language Support for Java(TM) by Red Hat > Main > main(String[])
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println("=== ДЕМО ООП ===\n");
4
5         // ПРАВИЛЬНЫЕ имена классов:
6         MountainBike mtb = new MountainBike(brand:"Trek", wheelSize:26, color:"Red", gears:27, suspension:true);
7         KidsBike kids = new KidsBike(brand:"Stels", wheelSize:16, color:"Blue", trainingWheels:true, maxAge:6);
8         BMX bmx = new BMX(brand:"GT", wheelSize:20, color:"Black", pegs:true, style:"Street");
9
10        // Инкапсуляция
11        System.out.println("1. ИНКАПСУЛЯЦИЯ:");
12        System.out.println("Старый цвет: " + mtb.getColor());
13        mtb.setColor(color:"Green");
14        System.out.println("Новый цвет: " + mtb.getColor());
15
16        // Наследование
17        System.out.println("\n2. НАСЛЕДОВАНИЕ:");
18        mtb.displayInfo();
19        kids.displayInfo();
20
21        // Полиморфизм
22        System.out.println("\n3. ПОЛИМОРФИЗМ:");
23        Bicycle[] bikes = {mtb, kids, bmx};
24        for (Bicycle bike : bikes) {
25            bike.ride();
26            bike.brake();
27            System.out.println("---");
28        }
29
30        // Абстракция
31        System.out.println("4. АБСТРАКЦИЯ:");
32        System.out.println("Все велосипеды имеют ride() и brake()");
33
34        // Счетчик
35        System.out.println("\n5. СЧЕТЧИК ОБЪЕКТОВ:");
36        System.out.println("Создано велосипедов: " + Bicycle.getCount());
37
38        BMX bmx2 = new BMX();
39        System.out.println("После создания BMX: " + Bicycle.getCount());
40
41        System.out.println("\n=== ВСЕ РАБОТАЕТ! ===");
42    }
43 }
```

Консоль рабочего кода:

```
=== ДЕМО ООП ===

1. ИНКАПСУЛЯЦИЯ:
Старый цвет: Red
Новый цвет: Green

2. НАСЛЕДОВАНИЕ:
Trek, колеса: 26, цвет: Green
Stels, колеса: 16, цвет: Blue

3. ПОЛИМОРФИЗМ:
Еду по горам на Trek (27 передач)
Торможу дисковыми тормозами
---
Детский велосипед Stels едет безопасно
Торможу ножным тормозом
---
Выполняю трюки в стиле Street на ВМХ
Торможу для трюка
---
4. АБСТРАКЦИЯ:
Все велосипеды имеют ride() и brake()

5. СЧЕТЧИК ОБЪЕКТОВ:
Создано велосипедов: 3
После создания ВМХ: 4

=== ВСЕ РАБОТАЕТ! ===
```

Контрольные вопросы:

1. Абстракция

Что: Выделение главных характеристик, скрытие деталей

Как: abstract class, interface, абстрактные методы

2. Инкапсуляция

Что: Соккрытие данных, доступ только через методы

Как: private поля + public геттеры/сеттеры

3. Наследование

Что: Создание новых классов на основе существующих

Как: class Child extends Parent

4. Полиморфизм

Что: Один интерфейс - разное поведение

Как: Переопределение методов (@Override)

5. Множественное наследование

Что: Наследование от нескольких классов

В Java: НЕТ для классов, ЕСТЬ для интерфейсов

6. Ключевое слово final

Для чего: Запретить изменение

Что можно: классы, методы, переменные

7. Модификаторы доступа

public - везде

protected - пакет + наследники

(default) - только пакет

private - только класс

8. Конструктор

Что: Специальный метод для создания объектов

Типы: По умолчанию, с параметрами, копирования

9. Ключевое слово this

Для чего: Ссылка на текущий объект

Использование: this.поле, this.метод()

10. Ключевое слово super

Для чего: Ссылка на родительский класс

Использование: super(), super.метод()

11. Геттеры и сеттеры

Что: Методы для доступа к private полям

Геттер: getПоле() - получить значение

Сеттер: setПоле() - установить значение

12. Переопределение

Что: Изменение реализации метода в классе-наследнике

Как: @Override public void метод() {...}

13. Перегрузка

Что: Методы с одним именем, но разными параметрами

Как: add(int a, int b) и add(double a, double b)

Вывод:

Все принципы ООП успешно реализованы и продемонстрированы в работе программы.